TREE DATA STRUCTURES

Short Notes • Exam Revision Edition

Formatted for clear reading, quick revision and printing

Important: The original short-note content has been retained. Only the page layout, typography, spacing and table presentation have been improved.

Basic Tree Terminologies – Short Notes

Here are all 16 terminologies in short, exam-revision format.

No.	Terminology	Short Definition
1	NULL Tree	A tree containing zero nodes.
2	Root Node	The topmost node of a tree; it has no parent.
3	Node	An individual element that stores data in a tree.
4	Parent Node	A node that has one or more children.
5	Child Node	A node directly connected below its parent.
6	Leaf Node	A node having zero children; also called a terminal node.
7	Non-Leaf Node	A node having at least one child; also called an internal node.
8	Degree of a Node	Total number of direct children of a node.
9	Degree of a Tree	Maximum degree among all nodes in a tree.
10	Level of a Node	Number of edges from the root to that node.
11	Height of a Tree	Maximum number of edges on the longest root-to-leaf path.
12	Path	A sequence of consecutively connected nodes.
13	Ancestor Node	Any node above a particular node on its path to the root.
14	Descendant Node	Any node reachable below a particular node.
15	Siblings	Nodes having the same parent.
16	Directed Edge	A connection directed from parent to child.
17	In-Degree	Number of incoming edges to a node.
18	Out-Degree	Number of outgoing edges from a node.
Terminology	Formula / Meaning	
Total Nodes	8	
Total Edges	Nodes − 1 = 7	
Degree of Node	Number of direct children	
Degree of Tree	Maximum degree of any node	
Level of Root	0	
Level of Child	Parent level + 1	
Height of Tree	Maximum level (when root is Level 0)	
Leaf Node	Degree = 0	
In-Degree of Root	0	
In-Degree of Other Nodes	1	
Out-Degree	Number of outgoing edges	

- Each node can have 0 to n children.
- No fixed maximum number of children.

2. Binary Tree

- Each node can have a maximum of two children.

- Left Child and Right Child.

Types of Binary Tree:

1. Standard Binary Tree: Normal binary tree with no special ordering rule.
2. Threaded Binary Tree (TBT): Uses threads in place of certain NULL pointers.
3. Binary Search Tree (BST): Smaller values on the left and larger values on the right.
4. AVL Tree: Self-balancing Binary Search Tree.

3. Binary Tree Operations

1. Creation
2. Traversal
3. Insertion
4. Deletion
5. Merge
6. Copy
7. Compare

Binary Tree Memory Representations

Two methods:

Representation	Explanation
Linked Node	Uses pointers (left, data, right).
Sequential Array	Stores nodes in a one-dimensional array.

- Left Child = `2*i + 1`
- Right Child = `2*i + 2`
- Parent = `floor((i-1)/2)`

2.4 Threaded Binary Tree (TBT)

Definition: A binary tree in which NULL pointers are replaced with threads pointing to inorder predecessors or successors.

Purpose: Traversal without recursion or stack.

Important fact:

- Total pointers = `2n`
- NULL pointers = `n + 1`

Three types:

Type	Thread direction
Left TBT	Inorder predecessor
Right TBT	Inorder successor
Double TBT	Both

Example tree:



Traversal	Rule	Output
Inorder	L–V–R	D B E A C
Preorder	V–L–R	A B D E C
Postorder	L–R–V	D E B C A

if (root != NULL)

{

```
    // Recursive calls and printf()
}
```

- Inorder: Left → Print → Right
- Preorder: Print → Left → Right
- Postorder: Left → Right → Print

Binary Search Tree (BST) – Short Revision

1. Definition

A Binary Search Tree (BST) is a Binary Tree in which:

- Left subtree contains smaller values.
- Right subtree contains greater than or equal values.
- Both subtrees follow the BST property.

2. Example

```
          50
         /  \\
       30    70
      / \\    / \\
&amp;#32;&amp;#32;&amp;#32;&amp;#32;20  40  60  80
```

3. Properties

- Maximum two children per node.
- Left values are smaller than the parent.
- Right values are greater than or equal to the parent.
- Every subtree must also follow the BST property.

4. Inorder Traversal

Rule: Left → Visit → Right (L–V–R)

Output: `20 → 30 → 40 → 50 → 60 → 70 → 80`

It produces sorted (non-decreasing) output.

5. Searching

1. Start from the root.
2. If equal → Element found.
3. If smaller → Move left.
4. If greater → Move right.
5. If NULL → Element not found.

6. Insertion

1. Start from root.
2. Compare the new value.
3. Smaller → Move left.
4. Greater → Move right.
5. Insert at the NULL position.

7. Deletion – Three Cases

Case 1: Leaf Node (No Children)

Simply remove the node.

Before: After:

```
   30           30
  / \\             \\
20 40     40
```

Case 2: One Child

Connect the parent directly to the node's child.

Before: After:

```
   50           50
  /            /
 30           40
   \\
&amp;#32;&amp;#32;40
```

Case 3: Two Children

Use either inorder successor or inorder predecessor.

- Successor: Smallest value in the right subtree.
- Predecessor: Largest value in the left subtree.

Example: Delete `50`.

Using successor `60`:

```
         60
        /  \\
      30    70
     / \\      \\
&amp;#32;20 40  80
```

Using predecessor `40`:

```
         40
        /  \\
      30    70
     /      / \\
20   60 80
```

8. Advantages

- Fast searching when balanced.
- Easy insertion and deletion.
- Inorder gives sorted data.
- Supports dynamic memory allocation.

9. Disadvantages

- Searching becomes slow if skewed.

- Requires extra pointer memory.
- Two-child deletion is more complicated.

10. Time Complexity

Operation	Average	Worst
Searching	O(log n)	O(n)
Insertion	O(log n)	O(n)
Deletion	O(log n)	O(n)

AVL Tree – Short Revision Notes

1. Definition

AVL Tree is a self-balancing Binary Search Tree (BST) in which the height difference between the left and right subtrees of every node is at most 1.

2. AVL Tree Rules

1. Must follow BST properties.
2. Every node must remain height-balanced.
3. Balance Factor must be `-1`, `0`, or `+1`.

3. Height Properties

- Leaf node height = `0`
- NULL node height = `-1`

Formula:

\[H(node)=1+\\max(H(L),H(R)) \]

Calculate height from bottom to top.

4. Balance Factor (BF)

Formula:

\[\\boxed{BF=H(L)-H(R)} \]

BF	Meaning
+1	Left subtree is taller
0	Both subtrees have equal height
-1	Right subtree is taller
+2	Unbalanced (left-heavy)
-2	Unbalanced (right-heavy)

5. Important Exam Formulas

Concept	Value
Leaf height	0
NULL height	-1
Node height	1 + max(L, R)
Balance Factor	Left Height - Right Height
Valid BF	-1, 0, +1
Unbalanced BF	Less than -1 or greater than +1

Remember:

- Height is calculated bottom to top.
- Balance Factor = Left − Right.
- Every node must satisfy the AVL balance condition.

Next Topic

AVL Tree Rotations

We will study the four rotations:

1. LL Rotation – Right Rotation
2. RR Rotation – Left Rotation
3. LR Rotation – Left-Right Rotation
4. RL Rotation – Right-Left Rotation

Case	Imbalance occurs at	Rotation required
LL	Left subtree of left child	Right Rotation
RR	Right subtree of right child	Left Rotation
LR	Right subtree of left child	Left, then Right
RL	Left subtree of right child	Right, then Left

AVL Tree – Rotations (Short Revision Notes)

1. Definition

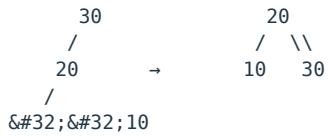
AVL Rotation is the process of balancing an unbalanced AVL Tree while maintaining the BST property.

When the Balance Factor becomes +2 or -2, rotation is required.

2. Four Types of AVL Rotations

1. LL Rotation (Left-Left)

- Insertion occurs in the left subtree of the left child.
- Requires a Right Rotation.



2. RR Rotation (Right-Right)

- Insertion occurs in the right subtree of the right child.
- Requires a Left Rotation.



```
      \\
&#32;&#32;&#32;&#32;&#32;&#32;&#32;&#32;30
```

3. LR Rotation (Left-Right)

• Insertion occurs in the right subtree of the left child.

• Requires two rotations:

1. Left Rotation
2. Right Rotation

```
     30              30              20
    /               /               /  \\
   10       →      20       →      10   30
     \\             /
&#32;&#32;&#32;&#32;&#32;&#32;&#32;&#32;20          10
```

4. RL Rotation (Right-Left)

• Insertion occurs in the left subtree of the right child.

• Requires two rotations:

1. Right Rotation
2. Left Rotation

```
 10                 10                 20
   \\                 \\               /  \\
    30       →         20     →      10   30
   /                    \\
&#32;&#32;&#32;20                      30
```

3. Important Exam Table

Rotation	Insertion position	Required rotation
LL	Left of left child	Right
RR	Right of right child	Left
LR	Right of left child	Left → Right
RL	Left of right child	Right → Left

4. Important Points

1. LL and RR require single rotations.
2. LR and RL require double rotations.
3. In double rotation, rotate the child first, then the unbalanced node.
4. Every rotation must maintain the BST property.
5. After rotation, the tree must satisfy the AVL balance condition.

Easy trick to remember:

• LL → Right

• RR → Left

• LR → Left, then Right

• RL → Right, then Left

AVL Tree – Insertion and Deletion Algorithms (Short Revision Notes)

1. AVL Insertion Algorithm

Definition: Inserting a new node using BST rules and balancing the tree if required.

Steps:

1. Insert the new node using normal BST insertion.
2. Update the height of each ancestor node.
3. Calculate the Balance Factor (BF).
4. Check whether any node is unbalanced (`BF > +1` or `BF < -1`).
5. Identify the rotation case (LL, RR, LR, RL).
6. Perform the required rotation.
7. Return the updated root.

Pseudocode:

AVL_INSERT(root, value)

1. If root == NULL:
   ```
   Create and return new node.
   ```
2. If value < root->data:
   ```
   Insert into left subtree.
   ```
3. Else if value > root->data:
   ```
   Insert into right subtree.
   ```
4. Update height of root.
5. Calculate BF of root.
6. If unbalanced:
   ```
   Identify LL, RR, LR, or RL.
   Perform required rotation.
   ```

```
7. Return updated root.
```

2. AVL Deletion Algorithm

Definition: Deleting a node using BST deletion rules and rebalancing the tree wherever required.

Three BST deletion cases:

1. Leaf node → Simply delete it.
2. One child → Replace the node with its child.
3. Two children → Replace with inorder successor or predecessor, then delete the replacement node from its original position.

Steps:

1. Search for the node using BST rules.
2. Delete the node using the three BST deletion cases.
3. Update the height of each ancestor node.
4. Calculate the Balance Factor (BF).
5. Check whether any node is unbalanced.
6. Identify the rotation case (LL, RR, LR, RL).
7. Perform the required rotation.
8. Continue checking ancestors up to the root.
9. Return the updated root.

Pseudocode:

AVL_DELETE(root, value)

1. If root == NULL:
   ```
   Return NULL.
   ```
2. Search for the node using BST rules.
3. Delete the node using BST deletion cases.
4. If root == NULL:

```
    Return NULL.
```

5. Update height of root.
6. Calculate BF of root.
7. If unbalanced:

```
    Identify LL, RR, LR, or RL.
    Perform required rotation.
```

```
8. Return updated root.
```

3. Insertion vs Deletion

Insertion	Deletion
Uses BST insertion.	Uses BST deletion.
Updates heights.	Updates heights.
Calculates BF.	Calculates BF.
Rebalances the first unbalanced ancestor.	May require rebalancing multiple ancestors.
Usually one rebalancing operation is sufficient.	May require multiple rotations.

4. Time Complexity

Operation	Time Complexity
Search	O(log n)
Insertion	O(log n)
Deletion	O(log n)
Single rotation	O(1)